

Shape w
Tell us w
under 1 r



2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: <https://dummyjson.com/user/me>
- En Authorization seleccione Bearer Token.

The screenshot shows a REST client interface with the following details:

- Request:** Method: GET, URL: <https://dummyjson.com/user/me>
- Authorization:** Bearer Token. The token field is empty.
- Response:** Status: 200 OK, Time: 67 ms, Size: 1.8 KB. The response body is a JSON object:

```
1 {
2   "id": 1,
3   "firstName": "Emily",
4   "lastName": "Johnson",
5   "maidenName": "Smith",
6   "age": 29,
7   "gender": "female",
8   "email": "emily.johnson@x.dummyjson.com",
9   "phone": "+81 965-431-3024",
10  "username": "emilys",
11  "password": "emilypass",
```

- Copie el accessToken obtenido en la solicitud Login.
- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.



GET `https://dummyjson.com/user/me` Send

Auth type: **Bearer Token**

Token: `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLT...`

Body: `{ "id": 1, "firstName": "Emily", "lastName": "Johnson", "maidenName": "Smith", "age": 29, "gender": "female", "email": "emily.johnson@x.dummyjson.com", "phone": "+81 965-431-3024", "username": "emilys", "password": "emilyspass", "birthDate": "1996-05-20"}`

200 OK • 50 ms • 1.81 KB

3. Comprobar qué ocurre sin autenticación

- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.
- Compare la respuesta con la solicitud autenticada.

GET `https://dummyjson.com/user/me` Send

Auth type: **No Auth**

No Auth

This request does not use any authorization.

401 Unauthorized • 610 ms • 1.12 KB

Body: `{ "message": "Access Token is required"}`



```
{  
  "message": "Access Token is required"  
}
```

Al no enviar el Bearer Token, la API no pudo identificar al usuario y rechazó la petición. En la solicitud autenticada la respuesta fue 200 OK con los datos del usuario; sin token, 401 y solo un mensaje de error.

4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.
- Indique qué función cumple el Bearer Token.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.

Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- a) Abra el proyecto C# de Azure Storage.
- b) En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- c) Busque la variable que contiene la Connection String.
- d) Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada:

AZURE_STORAGE_CONNECTION_STRING

La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

7. Crear la variable para ejecutar localmente

- a) En la carpeta principal del proyecto, cree un archivo llamado .env.
- b) Dentro del archivo agregue la variable:
AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING
- c) Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- d) Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.



```
MIA - AZURE BLOB STORAGE
=====
1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
=====
Seleccione una opción: 2

Archivos almacenados:
No hay archivos.

Presione ENTER para continuar...
█
```

- e) Compruebe que el programa continúa funcionando correctamente.
f) **No publique el archivo .env en GitHub**, ya que contiene la credencial real.

```
● >> git status --short
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it
M ../.gitignore
M ../Semana_12/MIA_AzureBlob/AplicacionAzure.cs
A AplicacionAzure.cs
A MIA_AzureBlob.csproj
A README.md
A prueba.txt
?? "../Semana_12/MIA_AzureBlob (2).zip"
?? "../Semana_12/MIA_AzureBlob.zip"
```

8. Crear el Secret en Github

- Ingrese al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: AZURE_STORAGE_CONNECTION_STRING
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.



Actions secrets and variables

Secrets and variables allow you to manage reusable configuration data. Secrets are **encrypted** and are used for sensitive data. [Learn more about encrypted secrets](#). Variables are shown as plain text and are used for **non-sensitive** data. [Learn more about variables](#).

Anyone with collaborator access to this repository can use these secrets and variables for actions. They are not passed to workflows that are triggered by a pull request from a fork.

Secrets

Variables

Environment secrets

This environment has no secrets.

Manage environment secrets

Repository secrets

New repository secret

Name

Last updated



AZURE_STORAGE_CONNECTION_STRING

now



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.